PROGRAMMING FUNDAMENTALS B16
SPRING 2025

Pointers

Dr. Ahmad Kazmi

Department of Computer Science

Faculty of Information Technology

Week 9 Lecture 25

Tue May 13, 2025

Outline

- A1, A2, Q1 & Q2 Marks are updated on Portal. Please review.
- Today
  - Variables and Pointer

Contains 7 Class Activities to be Completed handwritten and Submitted in the Next Class On Tue May 20

Variables and Pointers

Variables, their Memory Requirements and Addresses

1. int a = 5; // int: uses 4 bytes of memory
2. float b = 5.5; // float: uses 8 bytes of memory
3. char c = ‘a’; // char: uses one byte of memory
4. bool d = true; // bool: uses one byte of memory
5. double e = 5.5; // double: uses 16 bytes of memory
6. long f = 5; // long: uses 16 bytes of memory

Let’s Run this Code and check the Addresses and Confirm the Sizes

Switching to Compile and Run

Arrays, their Memory Requirements and Addresses

1. int a[2] = {1,2}; // 2 * int: uses 8 bytes of memory
2. float b[2] = {5.5, 6.6}; // 2 * float: uses 16 bytes of memory
3. char c[2] = {‘a’, ‘b’}; // 2 * char: uses 2 byte of memory
4. bool d[2] = {false, true}; // 2 * bool: uses byte of memory

Let’s Run this Code and check the Addresses and Confirm the Sizes

Switching to Compile and Run

Reminder about Variables

- All Variables have:
  - Name
  - Data Type
  - Value or Content
  - Address
- All Variables Exist in Memory i.e. No Memory No Variable
- All Variables have a Fixed Size Memory e.g. int 4 Bytes
- All Arrays have a Fixed Size (In [ ] during declaration)
- Address of a Variable Can’t be Changed
- A Variable is Created by the Compiler and Allocated Memory
- A Variable is only Deleted when Goes out of Scope: Out Side { }
- Address of a variable Can’t be Stored in Another Variable other than a Special Variable Called **Pointer**
- **To Understand Pointer, You Must Understand Computer Memory**

Computer Memory

Computer Memory Types and Usage

Two Main Types:

1. Read Only Memory (ROM): Written Once and Read Many Times
   - Retains Value, Even without Power
   - Contains BIOS: Code to Start the Computer and Load Operating System in RAM
2. Random Access Memory (RAM): Can be Written and Read many Times
   - Wipes Out without Power
   - After Startup of Computer by BIOS, the Operating Runs from RAM

RAM is of Two Types:

3. Code RAM: Contains all Executable Code
4. **Data RAM: Contains all Variables (Data)**

Data RAM is of 2 Types:

5. **Stack or Static Memory**: Contains all the Variables and Arrays
   - Much **Smaller** than Heap
   - Compiler **Allocates** (Creates), within { } and Deletes (Deallocates) when goes out of scope at the End }
     - **Compile Time Memory**
6. **Heap or Dynamic Memory**: Contains all the Dynamic Variables and Dynamic Arrays
   - Much **Larger** than Stack
   - Allocated (Created) and Deleted (Deallocated) Explicitly in Code Through **Pointers**
     - **Run-Time Memory**

Concept of a Pointer

Pointer: A Variable to hold an Address of a Variable of A Particular Data Type

```
int main()
{
      int x = 5;
      char word[10] = “Ahmad”;
    // x and word has an address allocated during Compile Time
    // Address of x and word cannot be changed
    // x and word Cannot be deleted in Code
    // x and word is Deleted at End Bracket } i.e. Goe out of Scope
    int * xp = &x; // a Pointer variable to hold address of data type int
                        // address of x is put in the pointer variable
    cout << xp << “ “ << * xp << endl;
     char * pWord = word; // or pWord = &word[0];
    cout << word << “ “ << pWord <<  “ “ << *pWord << endl;

  // Let us Run this Code, switching to Compile and Run
     return 0;
}
```

Pointer, Variable, Address

A Pointer is a variable to Hold Address of a Variable of a Particular Data Type

int a = 10;

int *pa = &a; // Pointer Variable pa is **Declared** and **Initialized** with the Address of Variable a

*pa = 20; // dereference pa → go to address in pa and set that memory to 20

char cstr[10] = “Test “;

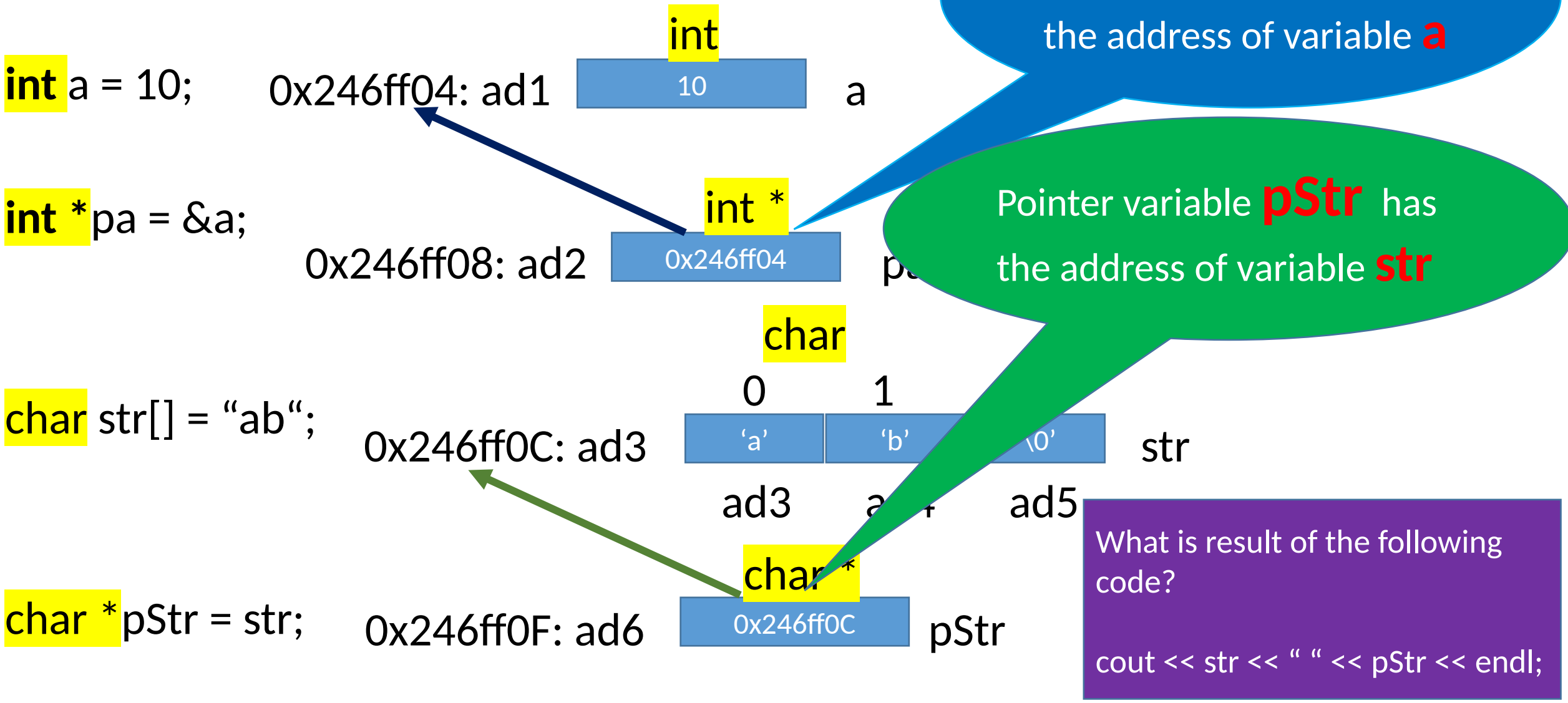
char *p = cstr; // Pointer Variable p is **Declared** and **Initialized** with the Address of array cstr

What is result of the following code?

cout << cstr << “ “ << p << endl;

Let’s run this code , switching to Compile and Run

Dry Run Example



Class Activity 1: Dry Run on paper and post the Scan in chat

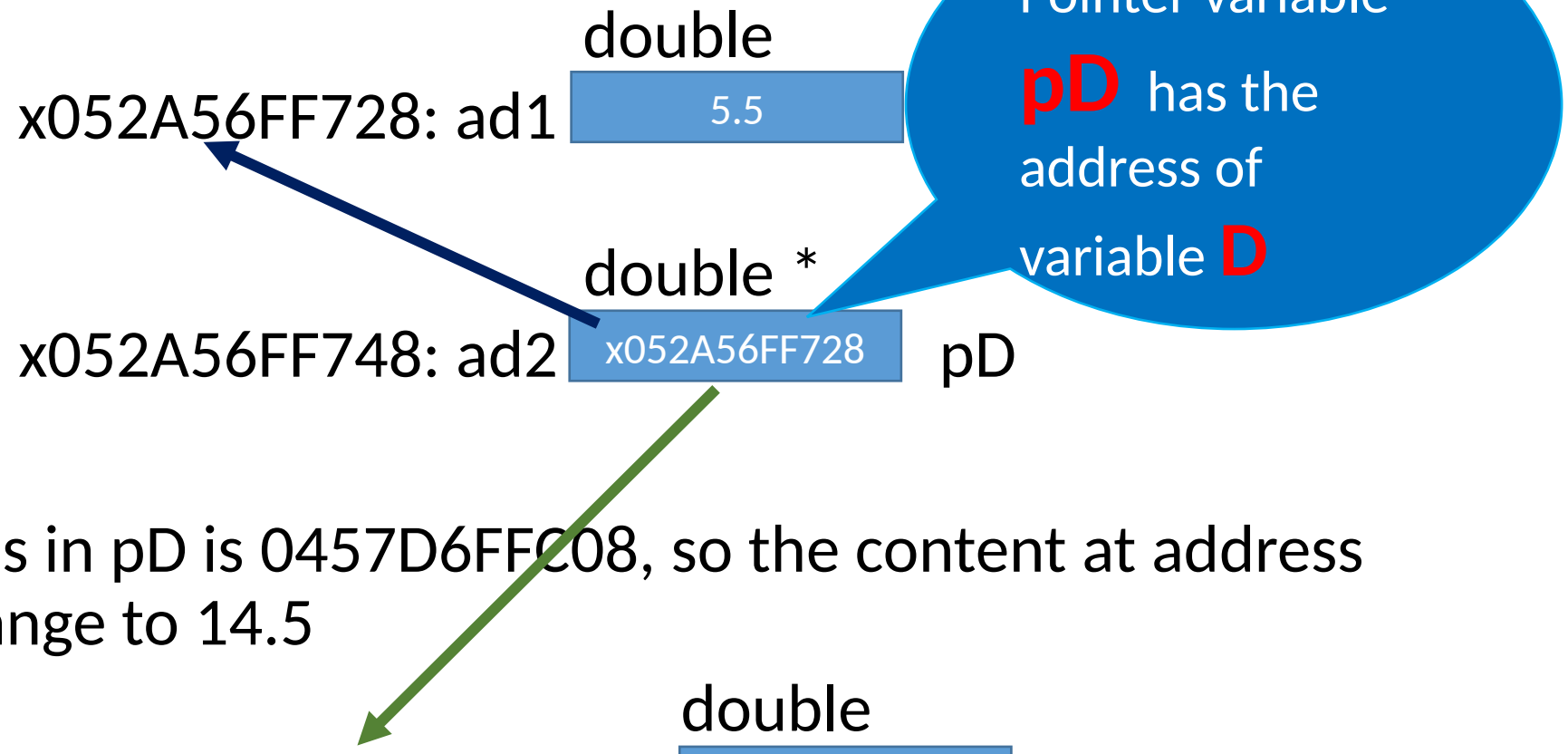
1. double D = 5.5;
2. double *pD = & D;
3. * pD = 14.5;
4. char word[] = “Ahmad“;
5. char *pWord = word;
6. cout << word << “ “ << pWord << endl;
7. pWord = & word[1];
8. cout << pWord << endl;
9. pWord = (word + 4);
10. *pWord = ‘Q’;
11. pWord[1] = ‘r’;
12. cout << pWord << endl;
13. pWord = (word + 6);
14. cout << pWord << endl;

Dry Run

1. double D = 5.5;

2. double *pD = & D;

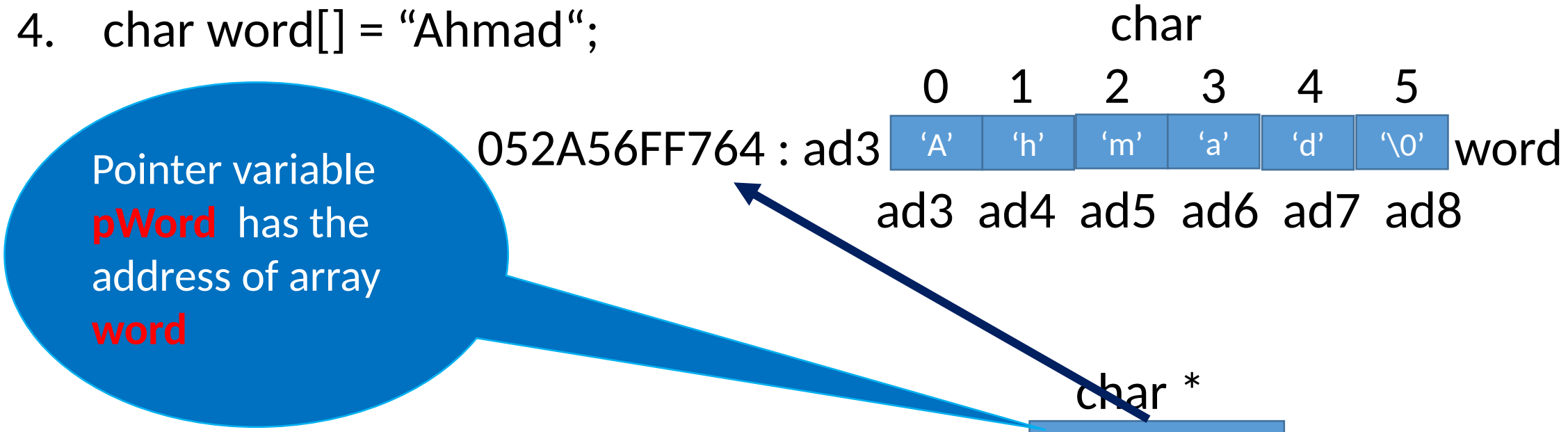
3. * pD = 14.5; // address in pD is 0457D6FFC08, so the content at address 0457D6FFC08 will change to 14.5



Pointer variable **pD** is dereferenced to reach address x052A56FF728

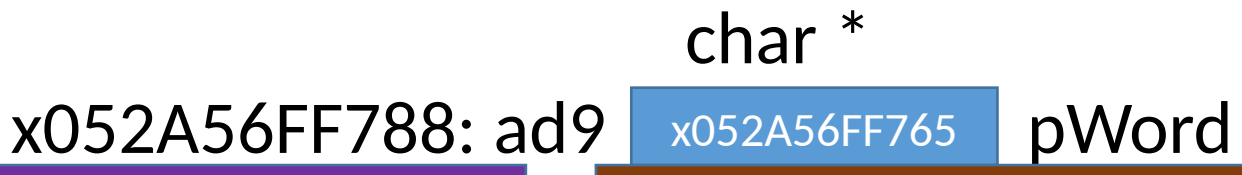
Dry Run

4. char word[] = “Ahmad“;



5. char *pWord = word;

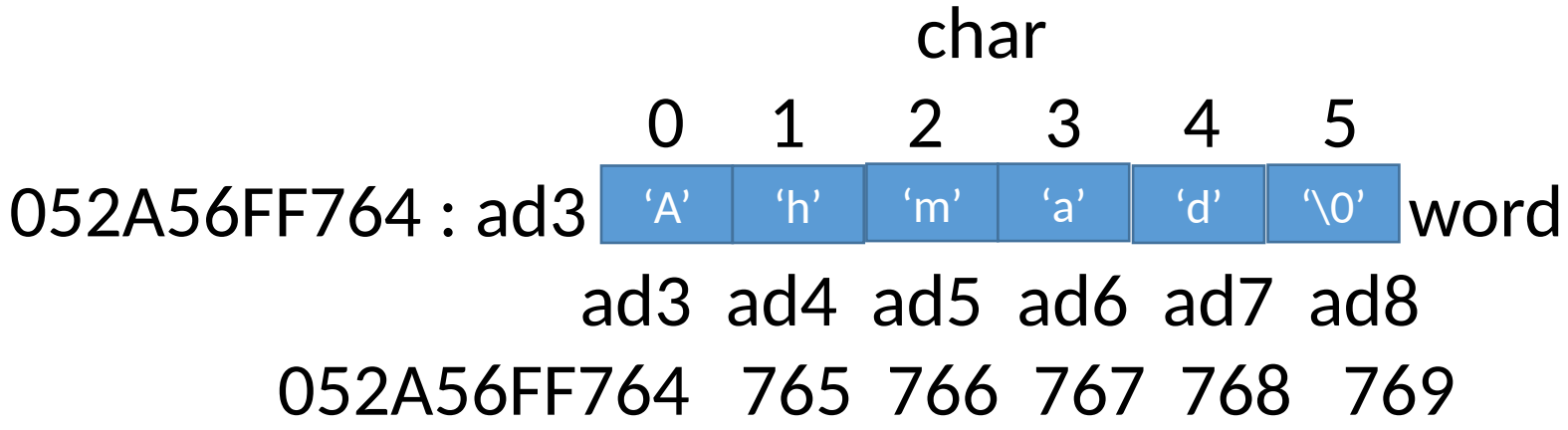
6. pWord = & word[1];



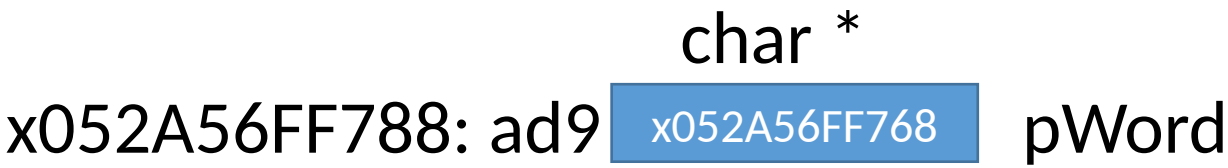
What is the address in pointer pWord?

Note the address of pWord
Why it doesn’t change?

Dry Run



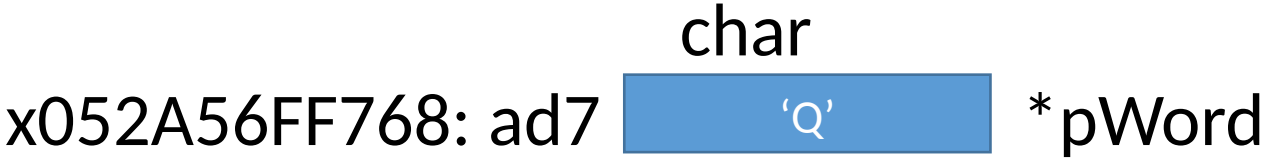
6. pWord = (word + 4);



What is the address in pointer pWord?

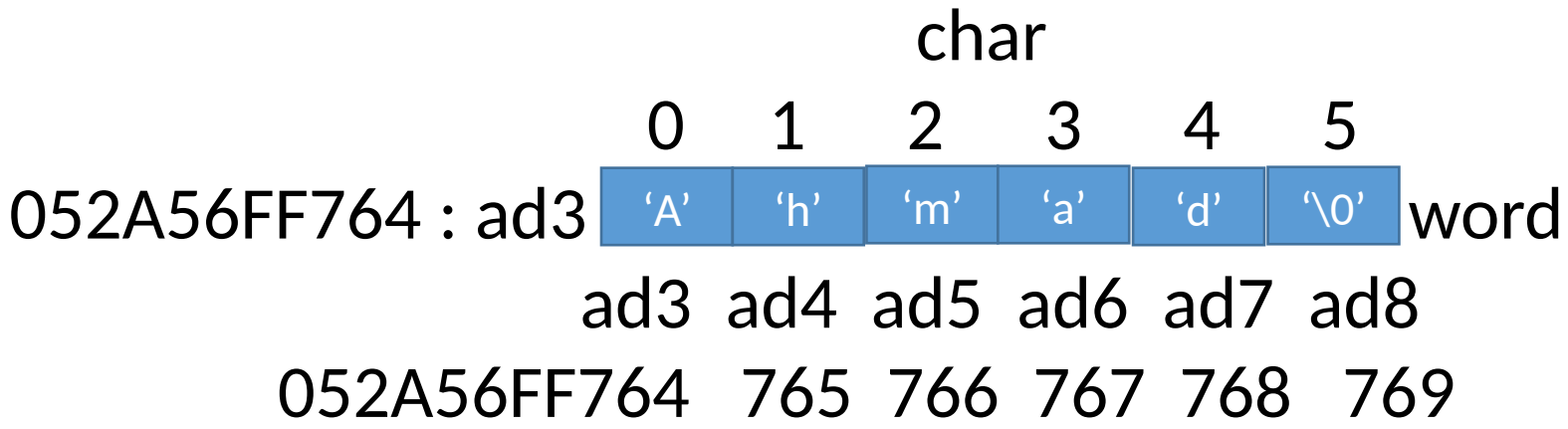
052A56FF764 + 4 = 052A56FF768

7. *pWord = 'Q';



Why the Value at Address x052A56FF768 is changed to 'Q'?

Dry Run

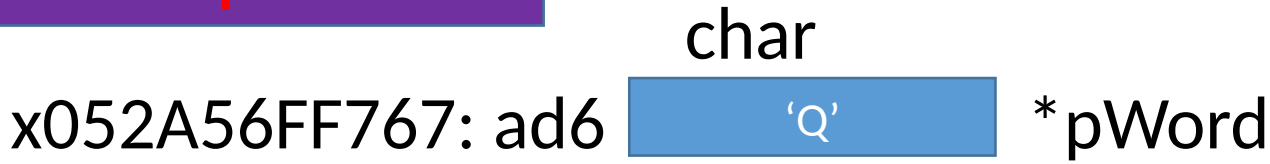


6. pWord = (word + 3);

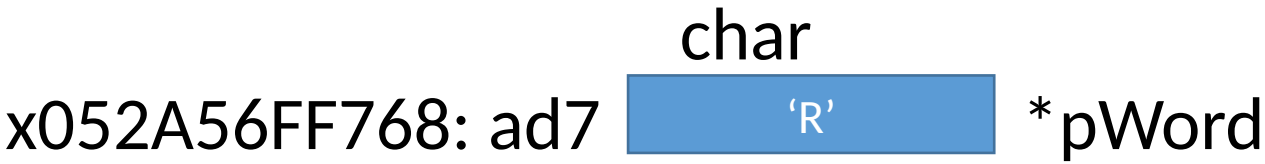
What is the address in pointer pWord?

052A56FF764 + 3 = 052A56FF767

7. *pWord = 'Q';



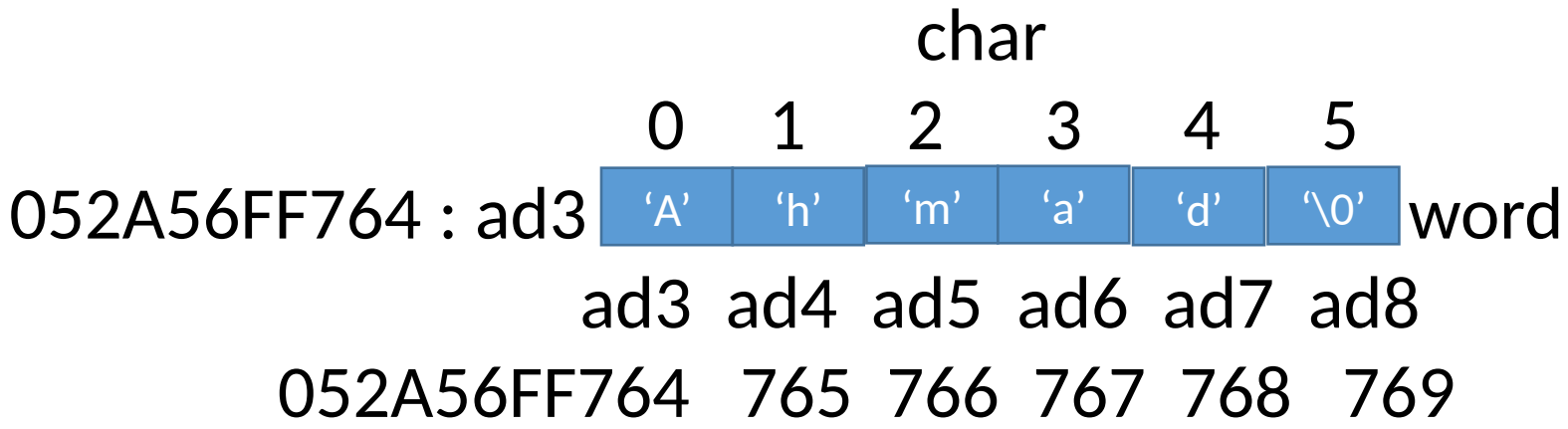
8. pWord[1] = 'R';



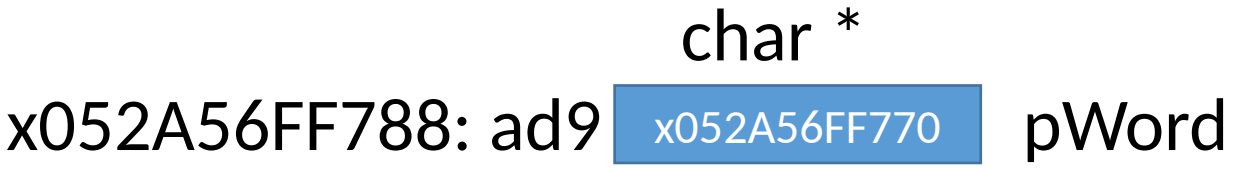
What is the address (pWord + 1) ?

052A56FF767 + 1 = 052A56FF768

Dry Run



6. pWord = (word + 6);



What is the address in pointer pWord?

052A56FF764 + 6 = 052A56FF770

7. *pWord = 'Q';

x052A56FF770

Why this line is a **run-time error?**

Learn Your Pointers

1. Pointers are Variables that Hold Addresses of Other Variables
   - A Pointer Hold A Single Address: A Positive Integer
   - Any Pointer Only Have a Single Address
2. Following Ways a Pointer Can be Assigned An Address:
   a. Of an Existing Variable using & Operator
      - int a = 10;
      - int * ip = & a;
   b. Of an Existing Array Allocated in Memory using the Array or an Offset from the Address of the Array
      - int array[5] = {1,2, 3, 4 ,5 5};
      - int * pArray = array;
      - Or
      - int pArray = (array + 2); // make sure that (array + 2) is within the bounds of the array
3. Never Assign an Integer value to a Pointer, Always use One of the Above Methods
4. Never Assign An Illegal or Undesirable Address to a Pointer
5. Pointers are Just Addresses E.G. Positive Integers
   - A Pointer can be Added, Multiplied or Divided by An Integer
   - But Make Sure that A Pointer Does Not Get An Illegal or Undesirable Address

Know Your Pointers

Always Draw And Know The Memory Allocation/Use Of A Pointer

- Int a = 10; // memory for 4 bytes or 16 bits allocated to a an int, with an initial value of 10

- int * ip; ; // memory for 8 bytes or 64 bits (size of Data of CPU) allocated to ip that will hold the address of an int variable. But currently garbage as no initial value

- ip = & a; // address of a is stored in the int pointer variable ip

- *ip = 20; // resolve the address stored in ip and the change the value at that address to 20

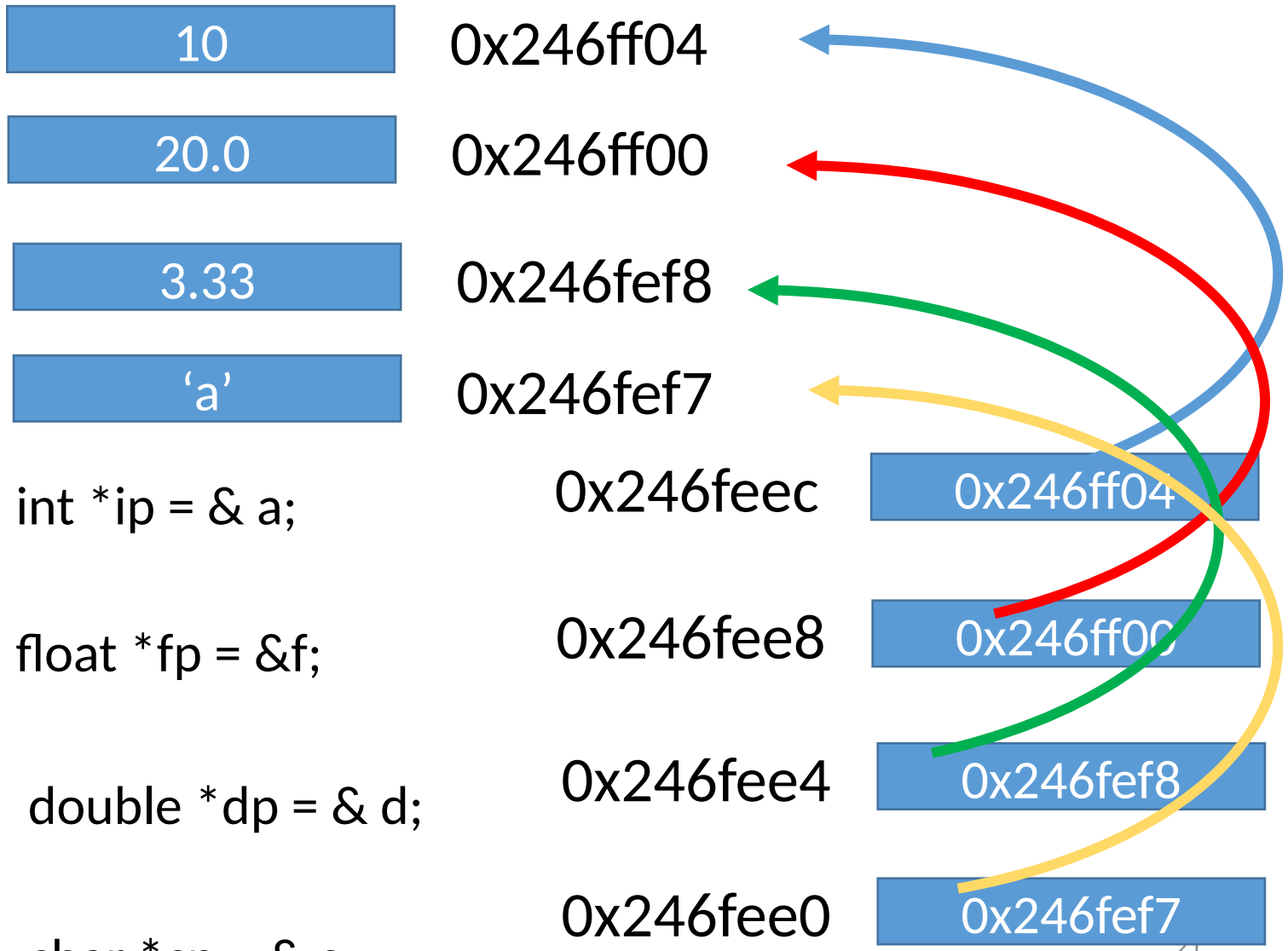
Pointer is a variable that stores address of memory of a particular data type

int a = 10;

float f = 20.0;

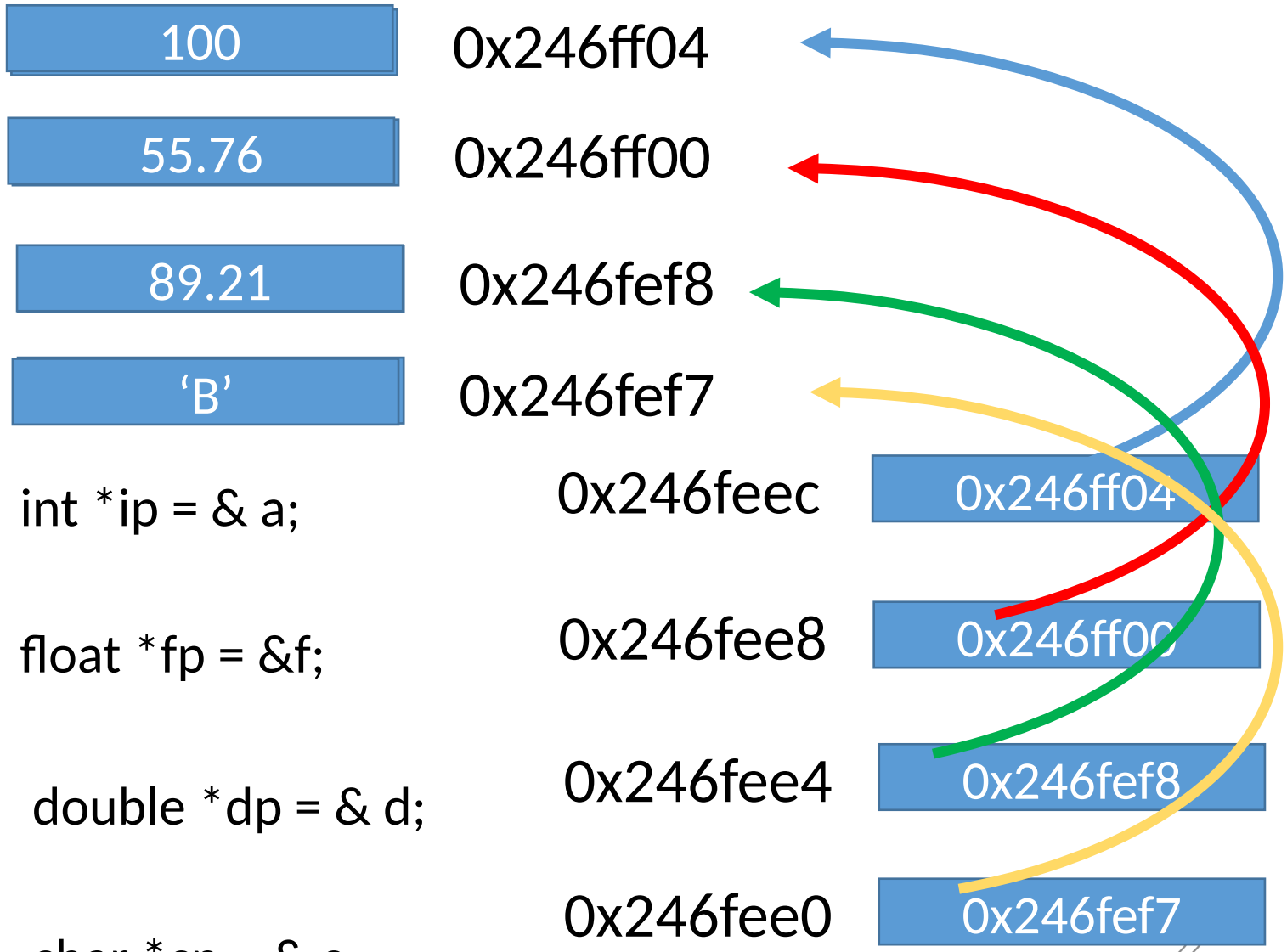
double d = 3.33;

char c = 'a';



Pointer is a variable that stores address of memory of a particular data type

```
int a = 10;

float f = 20.0;

double d = 3.33;

char c = 'a';
*ip = 100;

*fp = 55.76;

*dp = 89.21;

char *cp = 'B';
```



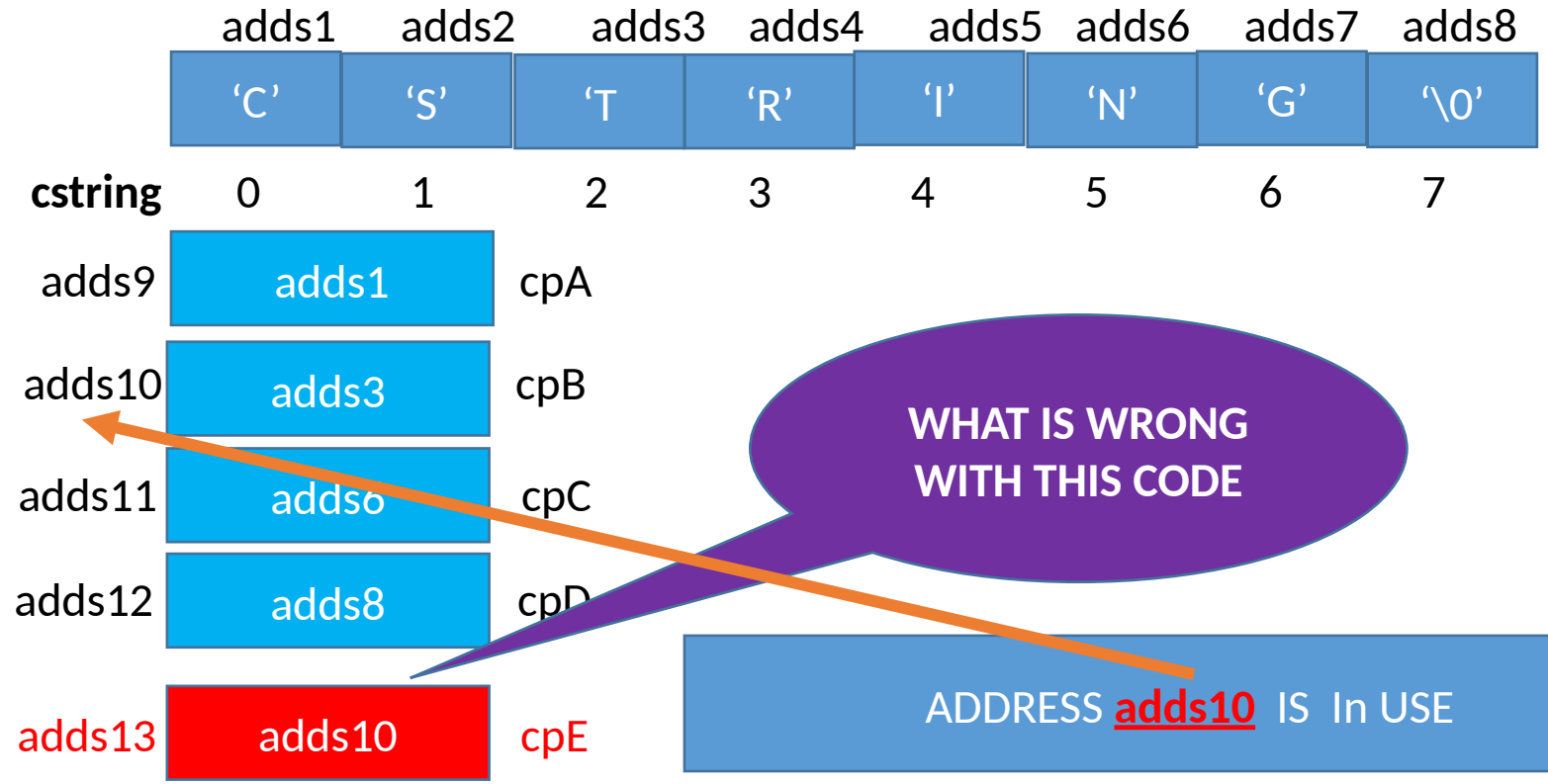
Pointer Basics: Arrays And Poir

- Arrays are Constant Pointers:
  - Assigned an Address Once, During Declaration Only
  - Can't Change the Address Stored in Array
- Pointers are Variables:
  - Can be Assigned an Address Any Time
  - Can Change the Address Stored in a Pointer
- Pointer Can be Assigned any Address of an Array:

char cstring[] = "CSTRING";

What Happen when the Following code is Executed:
*cpE = 'Q';
*cpB = 'W';

This will put 'Q' in Pointer cpB And 'Q" is not a Valid Address



char * cpA = cstring;

char * cpB = cstring + 2;

char * cpC = cstring + 5;

char * cpD = cstring + 7;

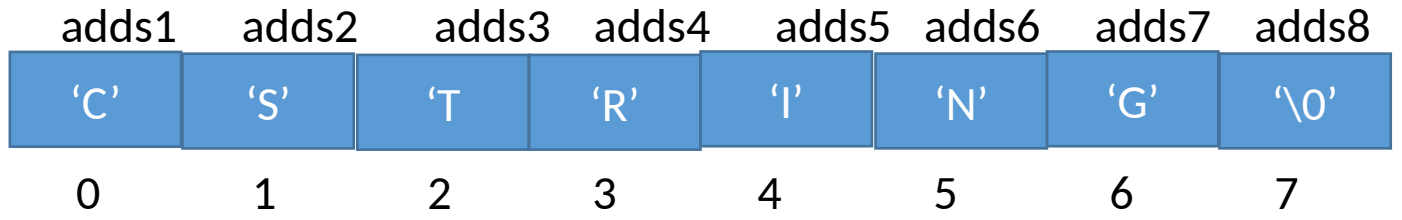
char * cpE = cstring + 9;

Pointer Basics: Using Pointers with Arrays

Once A Pointer is Assigned Address of an Array

It Can be Used Just As An Array

char cstring[] = “CSTRING”;



char * cp = cstring;

int length = 0;

while(cstring[length++] != ‘\0’) ; **or** **while(cp[length++] != ‘\0’) ;**

Memorize these Facts About Pointers

1. A Pointer is a variable that stores a **SINGLE** address of another variable
2. Each Pointer is a variable so It has its Own Address
3. The address Pointer Stores must be of the Same data type as indicated in the pointer declaration
4. During Declaration, A Pointer must be Initialized with Address of an Existing Variable or Array or with **nullptr (address 0x000000)**
5. **Never Assign a Variable Value or Constant to a Pointer, why? So the following code lines are wrong:**
   a) **int * p = 5**
   b) **int a = 5;**
      p = 5;
6. **Pointer Causes a Run-Time Error When Assigned an Illegal Memory Address**
7. **A Pointer Value is an Address and that is just a Large Positive Integer → A Pointer can be added, subtracted, multiplied, divided etc. as long as the resulting Value is a Address of a valid Memory**

Class Activity 2: Dry Run on paper and post the Scan in chat

```
1.  int main()
2.  {
3.    int values[5] = {1,2,3};
4.    int size = 3;
5.    int * pValues = values;
6.    for(int i = 0; i < size; i++)
7.     {
8.          values[i] = (i - values[i]);
9.     }
10.  for(int i = 0; i < size; i++)
11.   {
12.        pValues[i] = (i - pValues[i]);
13.   }
14. pValues = (values + 1);
15. for(int i = 0; i < size; i++)
16.   {
17.        pValues[i] = i;
18.        cout << pValues[i] << " ";
19.   }
20.  cout << endl;
21.  return 0;
22. }
```

Class Activity 3: Dry Run on paper and post the Scan in chat

```
1.  int main()
2.  {
3.    char STR[] = "abc";
4.    char * pSTR = STR;
5.    int length = 0;
6.    while(*pSTR != '\0')
7.      {
8.            length++;
9.            pSTR++;
10.     }
11.   cout << length << " " << pSTR << " " << STR << endl;
12.   return 0;
13. }
```

Class Activity 4: Dry Run on paper and post the Scan in chat

```
1. int main()
2. {
3.   int A[5] = {1,-2,3};
4.   int no = 3;
5.   int sum = 0;
6.   int * P = A;
7.   for(int i = 0; i < no; i++)
8.    {
9.         sum += *(P + i);
10.    }
11.  cout << sum << endl;
```

```
12. int min = A[0];
13.   for(int * I = A; I < (A + no); I++)
14.    {
15.         if(*I < min)
16.              {
17.                   min = *I;
18.               }
19.    }
20.   cout << min << endl;
21.   return 0;
22. }
```

Class Activity 5: Identify and Correct Errors, if any, in the Following Code Lines

1. int A = 10;
2. double B = 5.5;
3. int *P;
4. P = &B
5. double * Q = B;
6. int * R = & A;
7. R++;
8. double S = & B;
9. *S = 12.5;
10. S[1] = 12.5;
11. *(R + 1) = 5;
12. R[0] = 24;

Class Activity 6: What will be the Output of the following program

```
1.  int main()
2.  {
3.    char A[] = "abc";
4.    char B[5] = "";
5.    char * P = A;
6.    char * Q = B;
7.   while(*P != '\0')
8.   {
9.          *Q = *P;
10.        if(*Q >= 'a' && *Q <= ‘z’)
11.        {
12.                *Q -= 32;
13.            }
14.         cout << *P << " " << *Q << endl;
15.        Q++;
16.        P++;
17.    }
18.  cout << *P << " " << *Q << endl;
19.  cout << A << " " << B << endl;
20.  return 0;
21. }
```

Class Activity 7: What will be the Output of the following program

```
1.  int main()
2.  {
3.    char A[10] = "abc";
4.    char B[] = "def";
5.    int X = 0;
6.    while(A[X++] != '\0');
7.    char * Q = (A + X - 1);
8.    for(char * P = B; *P != '\0';P++, Q++,X++)
9.     {
10.          *Q = *P;
11.           cout << *Q << " ";
12.    }
13. cout << endl;
14.   A[X] = '\0';
15.   cout << A << endl;
16.  return 0;
17. }
```